

Documento derivato dall'analisi diretta del codice sorgente del progetto (non da specifiche esterne). La colonna **Stato** indica il grado di soddisfacimento nell'implementazione attuale: **Soddisfatto**, **Parziale**, **Assente**.

1. Prestazioni ed efficienza

ID	Requisito	Stato	Evidenza nell'implementazione attuale
PERF-01	Il server deve gestire richieste HTTP concorrenti tramite un pool di thread dedicato, senza bloccare le richieste in coda su un unico thread.	Soddisfatto	Pool fisso di 8 thread (<code>Executors.newFixedThreadPool</code>) impostato in <code>ServerMain</code> , dimensione non configurabile a runtime.
PERF-02	Le operazioni di lettura frequenti (elenco eventi, magazzino, pezzi liberi) devono rispondere senza elaborazioni superflue lato server.	Soddisfatto	Le API GET eseguono query dirette al DB e serializzazione JSON senza passaggi intermedi ne' cache; adeguato al volume dati previsto (progetto didattico).
PERF-03	Il sistema non deve introdurre colli di bottiglia sulle scritture concorrenti al database.	Parziale	SQLite consente un solo writer alla volta; le operazioni critiche (<code>creaMacchina</code> , <code>cambiaPezzo</code>) usano transazioni esplicite che serializzano gli accessi in scrittura, adeguato a bassa concorrenza ma non scalabile.

2. Sicurezza

ID	Requisito	Stato	Evidenza nell'implementazione attuale
SEC-01	Le credenziali degli addetti devono essere protette da accessi non autorizzati al database (hashing della password).	Assente	<code>ServizioAutenticazione.autentica</code> confronta la password in chiaro (<code>Addetto.getPassword().equals(...)</code>): nessun hashing (es. <code>bcrypt/argon2</code>) applicato.
SEC-02	L'accesso alle funzionalita' riservate deve richiedere una sessione valida con verifica del ruolo su ogni richiesta.	Soddisfatto	Decoratore <code>RuoloHttpHandler</code> + <code>ControlloAccesso</code> applicati a tutte le rotte sensibili (<code>/sessioni</code> , <code>/magazzino</code> , relative API).
SEC-03	L'identificativo di sessione deve essere non predicibile e trasmesso in modo da limitare l'esposizione a script lato client.	Parziale	UUID casuale, cookie <code>HttpOnly</code> ; <code>SameSite=Lax</code> ; manca l'attributo <code>Secure</code> (nessun enforcement HTTPS) e non e' previsto un timeout/scadenza della sessione.
SEC-04	Le richieste che modificano dati (form POST) devono essere protette da falsificazione di richiesta intersito (CSRF).	Assente	Nessun token CSRF nei form (login, prenotazioni, sessioni, magazzino).
SEC-05	Gli accessi al database devono essere protetti da SQL injection.	Soddisfatto	Tutte le query nei DAO SQLite* usano <code>PreparedStatement</code> con parametri bindati.
SEC-06	I tentativi di login ripetuti e falliti devono essere limitati per contrastare attacchi a forza bruta.	Assente	Nessun meccanismo di rate limiting o blocco account in <code>LoginHttpHandler</code> / <code>ServizioAutenticazione</code> .
SEC-07	I messaggi di errore restituiti al client non devono esporre dettagli implementativi interni.	Soddisfatto	Gli errori interni (HTTP 500) restituiscono messaggi generici; lo stack trace e' stampato solo su console server, non nel corpo della risposta.

3. Affidabilita'

ID	Requisito	Stato	Evidenza nell'implementazione attuale
AFF-01	Le operazioni multi-passo sul magazzino (assemblaggio macchina, cambio pezzo) devono essere atomiche: o si completano interamente o non alterano lo stato.	Soddisfatto	SqliteMagazzinoDAO.creaMacchina / cambiaPezzo usano transazioni esplicite (autoCommit=false) con rollback su qualunque eccezione.
AFF-02	Le operazioni su piu' entita' non transazionali (prenotazione/posti evento) devono mantenere la coerenza anche in caso di fallimento parziale.	Soddisfatto	SistemaPrenotazioni applica compensazione applicativa: se l'aggiornamento posti fallisce, la prenotazione appena inserita viene rimossa (e viceversa in fase di annullamento).
AFF-03	Gli errori nell'acquisizione di dati da fonti esterne non devono essere silenziati senza possibilita' di diagnosi.	Assente	GestoreEventiEsterni.caricaEventiDaCSVEsterno cattura IOException e la segnala solo con una stampa su console, senza log strutturato ne' propagazione.
AFF-04	Lo stato delle sessioni utente non deve essere perso in caso di riavvio pianificato del server.	Assente	Le sessioni sono mantenute in una ConcurrentHashMap in memoria (GestoreSessioniLogin): un riavvio invalida tutte le sessioni attive.

4. Disponibilita'

ID	Requisito	Stato	Evidenza nell'implementazione attuale
DISP-01	Il server deve poter essere arrestato in modo controllato senza perdita di richieste in elaborazione.	Soddisfatto	Shutdown hook registrato in ServerMain invoca BitFactoryWebServer.arresta(), che chiude il listener e termina ordinatamente il pool di thread.
DISP-02	Il sistema deve poter continuare a operare in presenza di guasto di una singola istanza (ridondanza).	Assente	Architettura a singolo processo, nessuna replica, nessun bilanciatore di carico o meccanismo di failover previsto.
DISP-03	Deve essere possibile verificare lo stato di salute del servizio senza eseguire operazioni di business.	Assente	Nessun endpoint di health-check dedicato; l'unico segnale di disponibilita' e' la risposta della home page.

5. Manutenibilita'

ID	Requisito	Stato	Evidenza nell'implementazione attuale
MAN-01	La logica applicativa deve essere separata su livelli distinti (presentazione, controllo, servizio, accesso ai dati, dominio).	Soddisfatto	Architettura a strati coerente: handler web -> controller -> service -> DAO (interfaccia + implementazione) -> model, in tutte le aree funzionali.
MAN-02	L'accesso ai dati deve essere disaccoppiato dal formato di persistenza tramite interfacce sostituibili.	Soddisfatto	Tutti i DAO sono definiti come interfacce (EventoDAO, MagazzinoDAO, ...); esistono gia' implementazioni alternative (CSV ed Sqlite per eventi/Legends).
MAN-03	I parametri di configurazione (porta, dimensione pool, soglie) devono essere esternalizzati, non cablati nel codice.	Assente	Porta (8082), numero di thread (8) e soglia di poca disponibilita' (2) sono valori letterali in ServerMain, nessun file di configurazione.
MAN-04	Le dipendenze fra componenti devono essere iniettate esplicitamente, evitando accoppiamento nascosto.	Parziale	Dependency wiring manuale via costruttori in ServerMain (nessun framework DI): chiaro per la dimensione attuale, poco scalabile se il progetto crescesse.

6. Portabilita'

ID	Requisito	Stato	Evidenza nell'implementazione attuale
PORT-01	Il sistema deve funzionare su qualunque piattaforma dotata di una JVM compatibile, senza dipendere da un application server esterno.	Soddisfatto	Basato esclusivamente su libreria standard (<code>jdk.httpservlet</code> , <code>java.sql</code>) e un solo driver esterno (<code>sqlite-jdbc</code>); nessun application server richiesto.
PORT-02	Il database deve essere trasportabile senza installazione di un DBMS server separato.	Soddisfatto	SQLite e' un database embedded su file singolo (<code>data/database_bfactory.db</code>).

7. Usabilita'

ID	Requisito	Stato	Evidenza nell'implementazione attuale
USA-01	L'utente deve ricevere un riscontro chiaro dell'esito di ogni operazione (successo o motivo del fallimento).	Soddisfatto	Messaggi di errore descrittivi (parametri, business rule violate) restituiti in pagina o come JSON <code>{"errore": ...}</code> .
USA-02	I form devono validare i dati lato client prima dell'invio, per ridurre andirivieni col server.	Assente	Il form sessioni e' marcato esplicitamente <code>novalidate</code> ; la validazione avviene interamente lato server.

8. Scalabilita'

ID	Requisito	Stato	Evidenza nell'implementazione attuale
SCAL-01	Il sistema deve poter aumentare la capacita' di elaborazione aggiungendo istanze (scalabilita' orizzontale).	Assente	Le sessioni di login risiedono in memoria di processo: senza sticky session o store condiviso, piu' istanze non condividerebbero lo stato di autenticazione.
SCAL-02	Il livello di persistenza deve poter sostenere un incremento del volume di scritture concorrenti.	Parziale	SQLite e' adeguato al carico di un singolo team/officina, ma impone un limite strutturale alle scritture concorrenti rispetto a un DBMS client-server.

9. Compatibilita'

ID	Requisito	Stato	Evidenza nell'implementazione attuale
COMP-01	Il codice deve essere compilato ed eseguito su una versione LTS recente del linguaggio, dichiarata esplicitamente.	Soddisfatto	Target Java 21 (<code>JPMS module-info.java</code> ; sintassi Java 21: switch expressions, text block, pattern matching su <code>instanceof</code>).
COMP-02	L'interfaccia web deve funzionare su un browser moderno standard, senza plugin aggiuntivi.	Soddisfatto	Pagine HTML/CSS/JS senza framework front-end ne' dipendenze plugin.

10. Vincoli tecnologici e di progetto

Vincolo	Descrizione
Linguaggio e piattaforma	Java 21 (JPMS), nessun framework applicativo esterno (no Spring/Jakarta EE).
Server HTTP	com.sun.net.httpserver.HttpServer (libreria standard JDK), non un server applicativo dedicato.
Persistenza	SQLite tramite driver xerial sqlite-jdbc 3.53.2.0; unica dipendenza esterna del progetto.
Concorrenza	Un solo processo JVM; pool di thread fisso configurato via codice, non a runtime.
Ambiente di build	Progetto Eclipse (.classpath/.project), nessuno script di build dichiarativo (Maven/Gradle) presente nell'archivio.
Distribuzione	Nessun artefatto di containerizzazione (Dockerfile) o script di deploy incluso nel repository.

11. Criticità prioritarie

Sintesi delle lacune piu' rilevanti individuate, in ordine di priorita' consigliata di intervento:

Priorita'	Criticita'	Requisito collegato
Alta	Password degli addetti confrontate e presumibilmente memorizzate in chiaro, senza hashing.	SEC-01
Alta	Assenza di protezione CSRF sui form che modificano dati (prenotazioni, sessioni, magazzino).	SEC-04
Media	Nessun rate limiting sul login: esposizione ad attacchi a forza bruta.	SEC-06
Media	Sessioni di login solo in memoria: perse ad ogni riavvio e non condivisibili tra piu' istanze.	AFF-04 / SCAL-01
Media	Configurazione (porta, pool, soglie) cablata nel codice: ogni modifica richiede una ricompilazione.	MAN-03
Bassa	Errori di importazione da fonti esterne segnalati solo su console, senza log strutturato.	AFF-03
Bassa	Nessun endpoint di health-check per il monitoraggio automatico.	DISP-03